

AYAWrus: An Educational and Machine Learning-Based Static Malware Detection System for CCS Students

A Capstone Project presented to the
College of Computer Studies of
Pamantasan ng Lungsod ng Pasig

In partial fulfillment of the requirements for the degree of
Bachelor of Science in Information Technology

By:

Cruz, Patricia Diane R.

Pineda, Karl Louis M.

Sta. Ana, Marc Andrew C.

Rodolfo A. Mirabel Jr., MT

Adviser

TABLE OF CONTENTS

CHAPTER 1.....	1
Introduction.....	1
The Problem and its Background.....	1
Review of Related Literature.....	3
Significance of the study.....	14
Statement of the Problem.....	16
Scope and Limitation of the Study.....	17
CHAPTER 2 : Methodology.....	21
Introduction.....	21
Research Design.....	22
Architectural framework.....	25
Level 0 Data Flow Diagram or schematic diagram.....	28
Algorithm Discussion.....	29
Software Engineering Paradigm.....	32
Tool to test for accuracy.....	39
Population and Sampling.....	40
Data Collection Methods.....	42
Data Analysis Procedures.....	43
Ethical Considerations.....	45
Reference List.....	47

CHAPTER 1

Introduction

This chapter of our study introduces the topic, and the problem and solution. It aims to outline the structure of what the study will contain. It contains the following section: Introduction, Review of Related Literature, Significance of the Study, Statement of the Problem and Scope and Limitation of the study.

The Problem and its Background

Incorporation of laptops into academic establishments has led to digital files becoming an important aspect of teaching, learning, programming, research, and administration. It is common practice for both students and professors to download source codes, software, projects, zipped files, and documents from different websites. Although this helps in academic pursuits, it can expose the laptop user to the threat of downloading malware posing as legitimate documents and software.

In the academic settings of Pamantasan ng Lungsod ng Pasig, especially students with laptops, students enrolled in computing related programs typically download development environments programming libraries, open source software and executable applications intended to be used as part of their course. And because these resources came from various online repositories and third-party sources, students may be exposed to malicious files or compromised executable files or ordinary files disguised as legitimate software, the laptops

being used for academic purposes may be susceptible to malware attacks. As soon as the infected file is opened, the malware can infect other devices, corrupt the files, steal confidential information, slow down the laptop, or even cause disruptions during laboratory work.

Even though conventional anti-virus software can be found in most of the laptops that have been infected by viruses, most of these laptops use signature-based detection. Signature-based detection involves comparing the scanned files with a database of the virus signatures that were detected previously. Even though signature-based detection is effective in detecting viruses that were detected previously, this process has shortcomings in detecting newly created, modified, and unknown malware.

While there have been increases in digital infrastructure utilization, there have been increasing cybersecurity problems. According to Zagita and Raharjo (2023), continuous use of technology increases the available attack surface for hackers. In the same way, Awasthi et al. (2024) highlighted that while interconnectivity of digital systems increases, adaptive and intelligent methods of protection are needed by organizations in protecting their data from any malicious attacks. This means that the existing traditional cybersecurity approaches are not enough in tackling malware issues.

To address such limitations, machine learning comes into play, which can be defined as a smart strategy to classify malware by analyzing its behavior and structure and not only by using pre-set signatures. Out of the many machine learning methods, kikelomo et al. (2023) have noted that the use of Random

Forest Classifier for the purpose of malware classification has provided an accuracy rate of 99.4%.

Recognizing these challenges, this study proposes the development of a Malware Detection System using the Random Forest Algorithm for the students of Pamantasan ng Lungsod ng Pasig. The system includes features such as file scanning, malware classification, scan history, quarantine management, file exclusion, allowed threats, model updating, and report generation to support monitoring and incident response.

Despite the increasing importance of cybersecurity education, learners lack the means to learn about malware detection by engaging in active and interactive learning experiences. Although various educational materials exist that cover the topic of malware and how it can be identified, there are few applications that use machine learning for malware detection purposes and that are targeted at students. There is a need for a user-friendly malware detection tool for the educational setting that will not only detect malware but teach about malware analysis and machine learning detection.

In addition to enhancing cybersecurity in students, the suggested system is also an educational tool that the College of Computer Studies can use. Through the use of the system, learners have the opportunity to learn how machine learning algorithms are used in malware detection and gain insights on topics like feature extraction, predictions, classification, and cybersecurity analysis.

The new system is not designed to replace any existing software; rather, it is meant to be used as an educational malware detection system that helps

students understand how machine learning can be applied to cybersecurity in practice. With the help of tools like malware classification, scan history, quarantine, confidence scores, and threats information, students will have the chance to learn more about malware detection in today's world as well as get acquainted with cybersecurity. The ultimate goal of the research is to develop an educational tool for the College of Computer Studies.

Review of Related Literature

Malware Threats and Detection

Malware threats have increased in both size and complexity, presenting major risks to individuals, organizations, and institutions that rely on digital systems. Traditional detection strategies, particularly signature-based approaches, have proven ineffective in dealing with these emerging threats. These systems operate by comparing files to a database of known malware patterns, which works well for familiar threats but fails completely when presented with new, updated, or obscured variations. This constraint has pushed academics to develop more adaptable, data-driven solutions.

Machine Learning Detection

Recent studies have shown that machine learning outperforms traditional detection approaches. Chaudhary et al. (2024) and Borate et al. (2024) both emphasize the increased effectiveness of machine learning in malware and adware detection, and the importance of broad datasets and strong techniques.

Their research involved different algorithms such as decision trees, Random Forest, Support Vector Machines, and Neural Networks. This is a demonstration of how machine learning algorithms can process large sets and detect patterns associated with malware. Combining supervised and unsupervised learning approaches, as well as advanced evaluation metrics like ROC-AUC and false positive rate, improves scalability and flexibility to evolving threats. These findings demonstrate that intelligent, pattern-based detection systems may identify sophisticated malware behaviors that signature-based techniques cannot.

Malware Classification

Malware classification is an essential component of modern incident responses. It enabled automated detection and threat prioritization for security and data analysts (Gundoor and Mulimani, 2024). Classification tasks are generally categorized into two primary types - binary classification and multi-class classification as experimented upon in a study by Chumachenko, 2017. Binary classification identifies a file as either malware or benign. Multi-class classification assigns a specific family label to a malicious sample(Ransomware, Worm, etc.)

Binary classification provides a straightforward foundation for malware detection because it focuses on determining whether a file should be considered malicious or benign. However, this binary system only provides simplicity and basic

knowledge. This kind of approach is particularly appropriate for systems whose primary objective is to identify potentially harmful files before further analysis is performed. Multi-class classification, in contrast, can provide more detailed and deeper insight such as distinguishing between ransomware, trojan, and other specific threats associated with a malicious sample. Although this helps increase understanding when analyzing the threats and responding to them, it requires datasets that are more specific and properly labeled. For an educational malware detection tool, binary classification is a good place to begin, as it enables students to learn about the fundamental machine learning process of distinguishing malicious files from legitimate files while maintaining a manageable classification scope.

Machine Learning and Random Forest

Among the machine learning approaches used in malware detection, the Random Forest algorithm has become known as one of the most reliable and extensively used classifiers. Random Forest trains by generating numerous decision trees, each based on a random subset of features and samples. This tree diversity enables the model to capture a wide range of malware features, such as API calls, code structure, and behavioral traces, while avoiding overfitting to specific malware families. The combination of these trees yields balanced, accurate classifications that routinely beat single-model techniques.

Recent studies have shown the effectiveness of this method across a variety of

deployment circumstances. Dhakal and Wan Fathul Hakim (2025) trained their Random Forest model on approximately 138,000 mixed samples and achieved around 99.5% accuracy, recall, and precision, proving that given enough data and appropriate feature selection, the algorithm may attain near-perfect performance. Similarly, Alsubaei et al. (2025) revealed that their adaptive Random Forest model detected new malware variants with approximately 96% accuracy and an F1-score of 0.96, demonstrating that the algorithm adapts efficiently to previously unknown threats.

Random Forest's effectiveness becomes more effective when static and dynamic features are included in the detection pipeline. Static measures like file signatures and code metrics capture structural malicious characteristics, but dynamic features like system calls and runtime behavior show how malware behaves during execution. Random Forest's flexibility and interpretability make it ideal for this hybrid approach; developers may track which features have the highest weight and alter extraction methods accordingly. Matsobane and Mokwena (2025) exhibited this adaptability by creating synthetic malware samples to balance an imbalanced dataset and retraining their Random Forest model to attain perfect recall and precision. Collectively these studies establish Random Forest not just as a high-performing classifier but as an adaptable, maintainable foundation for intelligent malware detection systems.

Cybersecurity Among Students

Research has found that despite students having a certain level of understanding about cybersecurity, many still find it difficult to identify cyber security challenges such as phishing attacks and malware among others. This problem makes a student, especially one not overtly familiar with basic computer knowledge, an appealing target for all types of malware: phishing, malware, viruses, ransomware, etc.. In some situations, students more often than not have proven to have basic understanding of preventative measures: scanning malware through antivirus software, scanning storage devices, deleting emails and installing trusted software (Funcion & Quisumbing, n.d.). However, the same type of consistency can not be achieved in phishing attempts. They understand how to achieve preventative measures but they do not understand the full scope of the danger of phishing and would often fall victim to it (Shahbazi et al., 2025).

This suggests that even though the students have basic knowledge about cyber security, there is a need for an effective educational approach which can facilitate the learning process for the students and enable them to learn about the methods of detecting malware in today's world. One study's result revealed that despite students' extensive and intensive use of the internet for academic and social purposes, they exhibit a noticeable lack of awareness regarding cybersecurity threats especially phishing attacks and password related attacks (Khaled et al., 2025). Even though students have basic knowledge about cybersecurity, research shows that they find it difficult to identify advanced cyber

threats. This suggests that there is a need for an accessible and interactive educational tool that allows students to explore modern malware detection techniques and to understand how machine learning can be integrated into modern malware detection.

Educational Cybersecurity

Despite the growing dependence of academic institutions on digital systems, cybersecurity education has struggled to keep pace with the demands placed on students. Goliath et al. (2026) surveyed undergraduate and postgraduate students and identified what they term a Cybersecurity-Resilience Gap, a measurable disparity in cybersecurity awareness and behavior that widens or narrows depending on students' stage in their academic careers, suggesting that earlier and more consistent intervention is needed rather than relying on students to develop awareness informally on their own. To address this gap, recent research points to explainability, giving users clear, understandable reasoning behind a system's output, as a key factor in improving both comprehension and trust among non-expert learners. Maity and Deroy (2024) argue that when AI-driven tools clarify the rationale behind their outputs rather than presenting a decision with no context, students are better able to understand the reasoning behind a result, engage critically with it, and retain the underlying concept, rather than simply accepting or dismissing the output outright. Paterakis et al. (2025), reviewing multiple user studies on non-expert interactions with machine learning systems, similarly found that users who were

shown the reasoning behind a model's output demonstrated improved understanding and calibrated trust, learning when to rely on a system's judgment and when to question it, compared to users given only a final result. Together, these findings support the value of systems that make their internal reasoning visible to the user, rather than functioning as an opaque, black-box tool, as a means of teaching students to understand the underlying logic of a security decision rather than simply memorizing its outcome.

Information Security

Information security, as a discipline, is broadly organized around the principles of confidentiality, integrity, and availability, commonly referred to as the CIA triad, which together define what it means for information to be adequately protected: that it remains accessible only to authorized users, that it remains accurate and unaltered, and that it remains available to those who legitimately need it. Vasović et al. (2026) argue that safeguarding the CIA triad has become critical for educational information systems specifically, noting that as academic institutions increasingly rely on interconnected digital platforms, including learning management systems, student records, and institutional repositories, the confidentiality, integrity, and availability of educational data becomes essential not only for regulatory compliance but for maintaining institutional trust and continuity of operations. This is not a purely theoretical concern; Seneviratne et al. (2022), in an empirical study of a higher education institution's information system, found that staff reported dissatisfaction across all three dimensions of

the CIA triad, with the institution's overall security posture assessed as highly vulnerable, suggesting that gaps in confidentiality, integrity, and availability are a documented and recurring issue within academic environments rather than a hypothetical risk.

Malware, as a category of threat, is directly implicated in this framework. Berrios et al. (2025), in a systematic review of malware detection and classification research, characterize malware as a persistent threat that specifically targets the confidentiality, integrity, and availability of information systems, whether through the theft of sensitive data, the corruption or unauthorized alteration of files, or the disruption of system access altogether. This framing positions malware detection not merely as a technical safeguard but as a mechanism that directly serves the broader goals of information security: identifying and containing malicious files supports confidentiality by preventing unauthorized data exposure, supports integrity by limiting unauthorized modification of system data, and supports availability by reducing the risk of system disruption. Understood this way, malware detection functions as one practical application of the CIA triad within an institutional setting, reinforcing why the development of accessible detection tools remains a relevant concern for academic environments specifically, rather than an isolated technical exercise.

Agile and ML Development Frameworks

Building an efficient malware detection system necessitates not only the correct algorithms, but also a disciplined development process that can handle

the iterative and exploratory nature of machine learning. Agile frameworks have developed as a realistic response to this difficulty. Vaidhyanathan et al. (2022) created Agile4MLS, a framework that tailors agile approaches to the unique needs of machine learning system development, emphasizing iterative experimentation and continuous process improvement. Ranawana and Karunananda (n.d.) presented the MLASDLC framework, which combines DevOps, MLOps, and SDLC principles to accelerate ML application development via continuous integration and parallel engineering. These frameworks show how hybrid development techniques that balance software engineering rigor with the unpredictable nature of machine learning are becoming increasingly important for developing dependable intelligent systems.

However, adopting agile approaches in security-focused systems introduces its own risks. Temitope and Kareem (2023) stress that the iterative nature of agile development can introduce vulnerabilities if cybersecurity risk management is not actively integrated throughout the process, recommending DevSecOps techniques and continuous risk assessment as countermeasures. Zagita and Raharjo (2023) further note that organizations under pressure to deliver products quickly often deprioritize non-functional requirements like information security, introducing risks to both reputation and operations. To address this, Dam et al. (2019) propose integrating artificial intelligence into agile project management to improve decision-making, automate repetitive chores, and boost security measures. Finally, aligning technical innovation with the growing requirement for

secure and adaptable development environments. Veido, Misnevs, and Plotkin (2019) and Nasiri and Lahmer (2024) demonstrate AI's ability to automate essential development tasks such as project success prediction and backlog refining, highlighting the need of intelligent tools in modern agile workflows.

ISO/IEC 25010 Software Quality

Evaluating the quality of a malware detection system requires a structured framework that goes beyond simple accuracy metrics. ISO/IEC 25010 provides this structure by defining software quality across multiple characteristics, each of which carries direct implications for how a system like a malware detector is designed, evaluated, and maintained.

Functional suitability is the first and most foundational characteristic, referring to the ability of software to provide functions that fully meet user needs. ISO/IEC 25010 breaks this down into three subcharacteristics: functional completeness, functional correctness, and functional appropriateness, which together ensure that the system covers all necessary functions, produces accurate results, and achieves user goals efficiently. Indykov et al. (2025) note that architectural decisions and tactics play a significant role in achieving these objectives, though each decision carries trade-offs that must be carefully considered. For instance, using ensemble tactics to improve detection accuracy may reduce false negatives but simultaneously increase false positives or raise resource utilization making empirical evaluation of each architectural choice essential.

Performance efficiency is equally critical, particularly for a system that must process files quickly under both normal and peak loads. This characteristic evaluates detection latency, throughput, and CPU and memory consumption, with benchmarks such as latency percentiles at p50, p95, and p99 on realistic workloads serving as practical indicators of operational efficiency. Indykov et al. (2025) provide a systematic basis for evaluating these metrics, having reviewed 206 studies to identify 11 quality attributes linked to 16 architectural tactics and 85 trade-offs, a framework that helps architects anticipate performance risks before they become operational problems.

Reliability in machine learning systems presents a unique challenge due to data variability and model uncertainty. The same review by Indykov et al. (2025) found that tactics like redundancy and monitoring can improve reliability but may affect other quality attributes in turn, reinforcing the importance of holistic design thinking rather than optimizing for a single characteristic in isolation.

Maintainability addresses how easily the system can be updated, patched, and improved over time with minimal disruption. Practical indicators include short update cycles, modular code architecture, and automated CI/CD testing pipelines all of which ensure the system remains effective as malware continues to evolve. Security, the final characteristic examined here, is defined under ISO/IEC 25010 as the system's ability to defend against malicious attacks and protect data

through confidentiality, integrity, accountability, and authenticity. For malware detection systems these principles translate into concrete safeguards such as encrypted model logic, secure update mechanisms, and resilience against evasion techniques. Siavvas et al. (2021) proposed a security model that evaluates internal security at both product and class levels, offering a structured approach to reinforcing system defenses. Complementing this, Sanju (2023) emphasized the importance of Intrusion Detection Systems as a key architectural tactic, particularly in dynamic environments where models must adapt to evolving behaviors and real-time data inputs. Studies by Liu et al. (2022a), Qu et al. (2017), Laqtib et al. (2019), Rashid et al. (2023), and El Balbali and Abou El Kalam (2023) further highlight the role of IDS in mitigating data poisoning and adversarial attacks, threats that exploit vulnerabilities in training data or input manipulation.

Implementation and User Guides

Beyond algorithm selection and quality evaluation, the practical implementation of a malware detection system requires comprehensive guidance that addresses both technical deployment and end-user accessibility. Fauzan et al. (2025) demonstrated the effectiveness of a web-based malware analysis system integrating Random Forest classifiers, highlighting the importance of implementation guides that cover web frontend development using frameworks such as Streamlit for rapid interactive prototypes and simplified user workflows. Alongside this, Manzoor and Arora (2023) proposed the concept of Malware-Aware Processors. hardware augmented with an online detector to

serve as a first line of defense reinforcing the need for implementation guides that ensure system adaptability to emerging and evolving malware threats.

Effective implementation guides must also provide detailed documentation for data preprocessing and feature extraction pipelines. Kikelomo et al. (2023) highlighted that a Random Forest classifier achieves greater accuracy compared to other classifiers, recording 99.4% accuracy, a result that depends heavily on standardized and reproducible feature extraction procedures. Dhakal and Wan Fathul Hakim (2025) further proposed the Gini method for creating split points, incorporating Gini Index and Gini Gain alongside bagging techniques to generate multiple decision trees from separate datasets, improving overall detection accuracy. Establishing standardized procedures for both static and dynamic feature extraction ensures consistency and reproducibility across different deployment environments.

Model training documentation is equally critical. Silva and Hernandez-Alvarez (2023) proposed deployment procedures for ransomware detection systems using dynamic analysis, including baseline Random Forest training recipes covering the number of trees and bootstrap sampling approaches to address class imbalance effectively. Reddy et al. (2023) further stress the importance of proximity metrics for anomaly detection, feature importance extraction, and tree-based explanation generation, frameworks that also incorporate model checkpointing, reproducible experiment logging, and robustness evaluation through adversarial perturbation testing and out-of-sample holdouts. Together

these studies establish that thorough training documentation is not optional but essential for operational decision making and long-term system reliability.

User accessibility is an equally important dimension of implementation. Fauzan et al. (2025) proposed using Streamlit to develop user-friendly interfaces that enable artifact submission and visualization of detection results with summaries accessible to non-expert users. Bejjam et al. (2023) further proposed deployment procedures for fileless malware detection systems utilizing memory forensics and containerized sandbox orchestration, practical considerations that user manuals must address to help users optimize the system across diverse computing environments.

Finally, long-term effectiveness requires comprehensive documentation for continuous performance evaluation and maintenance. Dhakal and Wan Fathul Hakim (2025) established benchmarks through standardized test suites including stratified 10-fold cross-validation, holdout set validation, and inference latency measurement to maintain high system accuracy over time. Kikelomo et al. (2023) proposed mathematical frameworks implementing automated drift detection to identify distributional changes, enabling operations teams to select efficient retraining schedules with documented threshold parameters and rollback procedures. Together these studies underscore that implementation guides must go beyond initial deployment addressing modular architectural design, user-friendly interfaces, and long-term maintenance protocols to ensure the

system remains effective against an ever-evolving threat landscape.

The literature reviewed across these themes builds a solid foundation for the development of the proposed educational machine learning-based malware detection system. Research on cybersecurity awareness among students highlights a persistent gap in students' ability to recognize modern threats such as phishing and malware, underscoring the need for more effective, accessible approaches to cybersecurity learning. Cybersecurity education research further supports the necessity of offering interactive learning experiences to students in order to help them understand the current threats and malware detection techniques. The Information Security literature complements this by grounding the study in the broader principles of confidentiality, integrity, and availability, establishing why malware detection matters as a form of protection in the first place, and not simply as a technical exercise. Addressing this need, research on the Random Forest machine learning model consistently points out the reliability and accuracy of this algorithm when it comes to classifying malware. The Agile and ML development frameworks literature then covers how such a system should be built, offering structured methodologies that balance the flexibility of machine learning with sound software engineering practices. ISO/IEC 25010 adds to this by providing a quality evaluation framework that ensures the system is not just accurate but also reliable, maintainable, and secure throughout its lifecycle. Finally the implementation and user guide literature connects research to practice, reinforcing that a well-built system is only useful if it can be properly

deployed and maintained by real users. In summary, these studies provide ample reasons for the creation of an educational malware detection system which not only demonstrates the practical implementation of machine learning in cyber security but also acts as another security measure for the students of College of Computer Studies.

Significance of the study

The development of the Malware Detection System using Random Forest Algorithm for the College of Computer Studies of Pamantasan ng Lungsod ng Pasig is expected to provide meaningful benefits to the following:

CCS Students

Students will be the main beneficiaries of this study. They will benefit from a direct interaction with an educational and working malware detection system. The system will provide explanations for how malware is typically scanned and processed by a system designed to detect Malware. Which is designed to help their understanding of cybersecurity concepts and develop technical skills in areas such as file scanning, feature extraction, and result analysis. The system provides a hands-on learning experience (Machine Learning Training and Updating, and Malware Diagnostics and Analysis) that reinforces what is taught in the classroom and prepares students for real-world cybersecurity challenges.

Pamantasan ng Lungsod ng Pasig - Management Information System

The institution, specifically the Management Information System (MIS) Office, may directly benefit from this study by having access to an educational malware detection system that may support cybersecurity awareness initiatives within the university. The system may be used as a supplementary tool to demonstrate modern malware detection techniques and to encourage responsible cybersecurity practices among students. In addition, the system provides malware scanning, quarantine management and incident reporting contributes to additional security capabilities. These features contribute to a more secure and safe environment.

The Researcher

The development of this education malware detection system will enhance the researcher's understanding of how students can learn and understand cybersecurity. It will also allow the evaluation of the students' technical knowledge and skills in machine learning, cybersecurity and system development. The process of designing, building and evaluating an education malware detection system will provide technical knowledge and understanding in machine learning training, malware handling and disposing, and malware diagnostics and analysis - competencies that are valuable both academically and in everyday life

Future Researchers

This study can serve as a reference and foundation for future research in educational malware detection and machine learning applications. Future researchers may build upon this system by expanding its dataset, incorporating additional algorithms, improving the user interface, or extending its scope beyond the students of Pamantasan ng Lungsod ng Pasig. The system's modular design and documented methodology provide a solid starting point for further development and improvement for a more enhanced educational system.

Statement of the Problem

In view of the proliferation of malware attacks in both educational and corporate computing environments, the College of Computer Studies of Pamantasan ng Lungsod ng Pasig is investigating the creation of a Malware Detection System utilizing the Random Forest Algorithm. This study intends to create a more intelligent, adaptable, and effective cybersecurity solution for the use of students and faculty members.

1. What Malware Detection System using the Random Forest Algorithm can be developed for the College of Computer Studies?
2. What features can be incorporated into the proposed Malware Detection System?
3. How is the performance of the Malware Detection System with respect to its functional suitability, performance efficiency, reliability, maintainability,

and security according to the ISO/IEC 25010 standards of software quality?

4. What implementation guide can be prepared for the proposed Malware Detection System?
5. What user manual could be developed to help CCS students use the system?

Scope and Limitation of the Study

This research aims to investigate the development of AYAWrus: A Static Malware Detection System for the Students of College of Computer Studies . This system is expected to aid the MIS Office through an intelligent static malware detection system which can analyze the Window PE file without executing it. The Random Forest Algorithm is used as the machine learning classifier of this system, which can detect the malicious or benign files of an executable file based on its static characteristics.

The scope of this study includes the following functional aspects of the system:

Random Forest Classification Model. In the proposed system, there will be a trained Random Forest Classifier that serves as a malware classification model for the system. It is trained using the EMBER 2024 dataset that was created by Robert J. Joyce. This dataset contains labeled samples of Windows PE files. The trained model will be used for classifying executable files as benign or malicious.

Static File Scanning. The system uses static analysis to scan Windows PE files including those in .exe, .dll, .sys, .ocx, and src file formats. The system analyses the extracted file features without executing them. Users may use any of the following scans depending on their needs - Quick Scan, Full Scan or Custom Scan.

Quarantine Management. Malicious files are moved to the quarantine folder where they can either be analyzed further, deleted permanently, or restored if they are found to be false positives.

File Exclusion Management. Users can mark certain trusted files or directories as exempt from being scanned in subsequent operations so as to reduce the chance of false positives.

Scan History and Incident Logging. The system maintains a history of all scan operations carried out including any detected malware and the actions taken by the user during scanning.

Threat Information. For each identified threat file, the system shows the file's predicted classification, confidence, and recommended actions taken by the users.

Model Updating. The system supports the updating functionality that allows updating the trained machine learning model when a new one is released with any changes to the application itself.

However, the mentioned functionality is still associated with some drawbacks.

The system uses only static malware analysis without performing any dynamic or behavioral analysis and process monitoring. Static Characteristics of Window Portable Executable files are evaluated.

The performance of the Random Forest classifier is very much dependent upon the quality of the EMBER 2024 dataset that is used for training the machine learning model. Given the fact that the machine learning algorithm is based on publicly available datasets, it is about to encounter false positive and false negative cases and also struggles to detect novel zero-day, highly obfuscated, polymorphic, and anti-machine learning malware.

The system is confined to only Window Portable Executable (PE) files and will not be able to perform analysis on any other file format such as PDF, Microsoft Office files, images, videos, archives, scripts, and mobile applications. Similarly, the system can only be used in the Microsoft Window platform and cannot be used in any operating systems.

The scanning process can be affected by the laptop's hardware capabilities and the number of files to be scanned, as well as their size. Scanning large files may take more time than scanning a small file.

One limitation of the system is that it does not classify detected threats into specific malware families or types, such as **Trojans, ransomware, spyware, adware, worms, rootkits**, or other malware categories. Instead, the system focuses on determining whether a file is malicious, suspicious, or clean based on the machine learning model's prediction. As a result, while the system can effectively detect potentially harmful files, it does not provide detailed information about the exact type or family of malware detected. This limits the level of threat intelligence available to users and security analysts and may require additional analysis using specialized malware classification tools.

Additionally, the system's current output for a detected threat is limited to a predicted classification, confidence score, and recommended action, as outlined in the Threat Information feature above. It does not yet provide a detailed, feature-level explanation of the specific characteristics or behaviors that led to a file being classified as malicious or benign. Future development of the system may extend this functionality to support fuller transparency into the reasoning behind a classification, in line with the system's educational goals.

Lastly, AYAWrus is created mainly for educational and institutional use by the students of the College of Computer Studies. The system shows how machine learning can be applied in static malware detection and acts as an additional security measure. The tool cannot be used as a replacement for professional antivirus software created by cybersecurity companies.

CHAPTER 2 : Methodology

This chapter presents the methodology and the framework of the proposed system. It discusses the concept of the study, the system's conceptual operation. It also includes the mockup and the methodological approach used in developing and evaluating the system.

Introduction

The purpose of this chapter is to present the assumptions, methods and techniques used in the development of the **Malware Detection System**. It aims to provide a clear and systematic description of how the study was conducted to ensure that the research can be replicated.

Specifically, this chapter discusses the research design, the implementation of the **Random Forest Algorithm** and its application in detecting and categorizing malicious and benign files as well as first and foremost, as an educational tool. It will also describe the datasets applied in the research including the process of data collection, preparation, and training of the model.

Furthermore, this chapter outlines the development of a malware detection system utilizing the Algorithm to classify files as either benign or malicious. This system will focus on analyzing common file types and documents by identifying patterns associated with malware using machine learning tools.

Research Design

This study employs a quantitative experimental research design in evaluating the effectiveness of implementing Random Forest Algorithm into a Malware Detection System. The quantitative approach was utilized as the study relies on numerical data and performance metrics such as accuracy, precision, recall and F1 - score to assess the model's performance.

To begin with, Random Forest was chosen as the machine learning model to be implemented for the Malware Detection System. As an ensemble method, Random Forest is an effective and efficient way to classify malware. It uses multiple decision trees for robust classification and reduces overfitting. Kikelomo et al. (2023) demonstrated that a Random Forest Classifier generates greater accuracy outperforming KNN, Decision Tree, Logistic Regression, and other algorithms with an accuracy of 99.4%. Similarly, Garcia and Muga (2016) highlighted the effectiveness of Random Forest to achieve high predictive accuracies with minimal parameter tuning compared with Gradient Boosted Trees, Neural Networks, and Support Vector Machine.

The experimental design involved training the Random Forest Algorithm utilizing the EMBER 2024 Dataset by Joyce et al (2024), a large-scale benchmark dataset of Portable Executable (PE) Files of approximately 3.2 million files developed for malware classification. For this study, due to computational resource constraints, a representative subset of the dataset was utilized while

preserving the official training, testing, and challenge subset for model development and evaluation.

To evaluate the performance of the Random Forest model, the dataset partitions were utilized together with an internal validation procedure. The training subset was first preprocessed by removing unlabeled samples. It was then divided into training and validation sets using an 80 - 20 stratified split to preserve the distribution of malware and benign samples. The training set was used to train the Random Forest Classifier while the validation set was used for hyperparameter optimization and threshold selection

Hyperparameter tuning was performed using the Optuna optimization to identify the optimal Random Forest parameters that maximizes the model's performance. Following optimization, the final Random Forest model was trained using the selected hyperparameters and evaluated on the test set which consists of previously unseen samples. This kind of unbiased approach allowed the model's ability to generalize to new executable files. In addition to this. The trained model was evaluated on the challenge set to examine the performance on additional malware that initially went undetected by all antivirus engines.

The performance of the proposed malware detection system was measured using the standard classification metrics, including accuracy, precision, recall, F1-score, and the Area Under the Receiver Operating Characteristic Curve

(ROC-AUC). Confusion matrices, classification reports and feature importance analysis were also generated to provide a comprehensive evaluation of the Random Forest classifier.

This research design is appropriate for this study because it evaluates the interaction of machine learning into a malware detection system using objective and reproducible methods. It allows for objective measurement of the system's performance and determines the effectiveness and efficiency of the proposed machine learning approach in malware detection. To ensure that the results are reproducible and reliable, statistical and reproducible, statistical evaluation methods were employed. The model was evaluated using the test and challenge set to reflect real-world malware classification performance.

Architectural framework

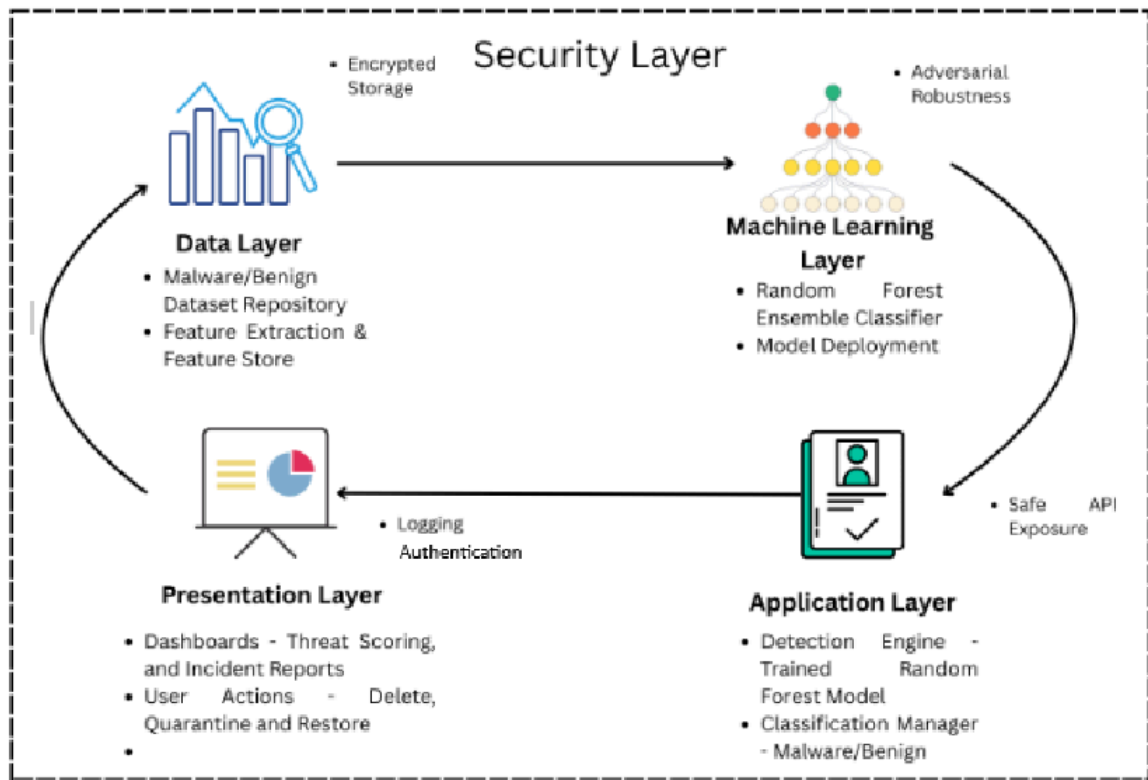


Figure 1. Data Flow Diagram

The figure above demonstrates the architectural framework of our systems. It contains four(4) layers: Data Layer, Machine Learning Layer, Application Layer, Presentation Layer. It also contains one(1) cross-cutting layer. Each layer is connected to each other through the feedback loop. It starts with the Data Layer.

The Data Layer contains necessary information and data to train the machine learning model. It includes the malware/benign dataset repository for the model to use as samples. As well as the feature extraction and feature stores. This is done in order to extract valuable information and characteristics that the model

can read. The Data Layer is critical as this is where the foundation for a good model starts. Good Feature determines accuracy and Data Integrity is important otherwise the data that will be used by the model will be poisoned or skewed.

The Machine Learning Layer contains the model itself that will be used for malware detection systems. The Random Forest Ensemble Classifier Model uses multiple decision trees to classify samples or data as benign or malicious. Ensemble increases the rate of accuracy and reduces overfitting. The Trained model will be exposed to the application layer for real-time classification of malware. The Machine Learning Layer is important as this is the part where the model is trained and re-trained to account for future malware.

The Application Layer is where the execution of logic in real-time takes place. It contains the Detection Engine where it runs the model in production to detect and classify the malware through the classification manager. The Classification Manager manages the result from benign to malware and coordinates with the quarantine control system.

The Presentation Layer provides the UI and UX that will be shown to the user. It contains the dashboard: threat scoring, incident report, real-time detection stats. As well as other various user actions where the user operator can delete suspicious files, quarantine or restore false positives or false negatives for testing purposes.

After all that, a continuous feedback loop connects all the layers together. New Data and samples received from the Presentation layer is fed back to the data layer for an iterative process. This iterative process ensures that the model can be re-trained using the updated data to account for future malware. Ensuring that the model can be trained to reach its optimal performance and reducing false positives and false negatives.

To account for security, each layer is coated in a protective umbrella. The security layer is the cross-cutting layer to protect and prevent malware from compromising or attacking our system. Each four(4) layer is surrounded with their own security. Encrypted Storage ensures the dataset is protected. Adversarial Robustness ensures the model is protected from adversarial attack - malware slightly altered to evade detection. Safe API Exposure ensures the detection engine or classification is protected from outside interference. Logging and Authentication ensures that every action is logged, prevents misuse and helps in incident response. Each Security Component is important as they protect the system from being overrun or compromised.

Level 0 Data Flow Diagram or schematic diagram

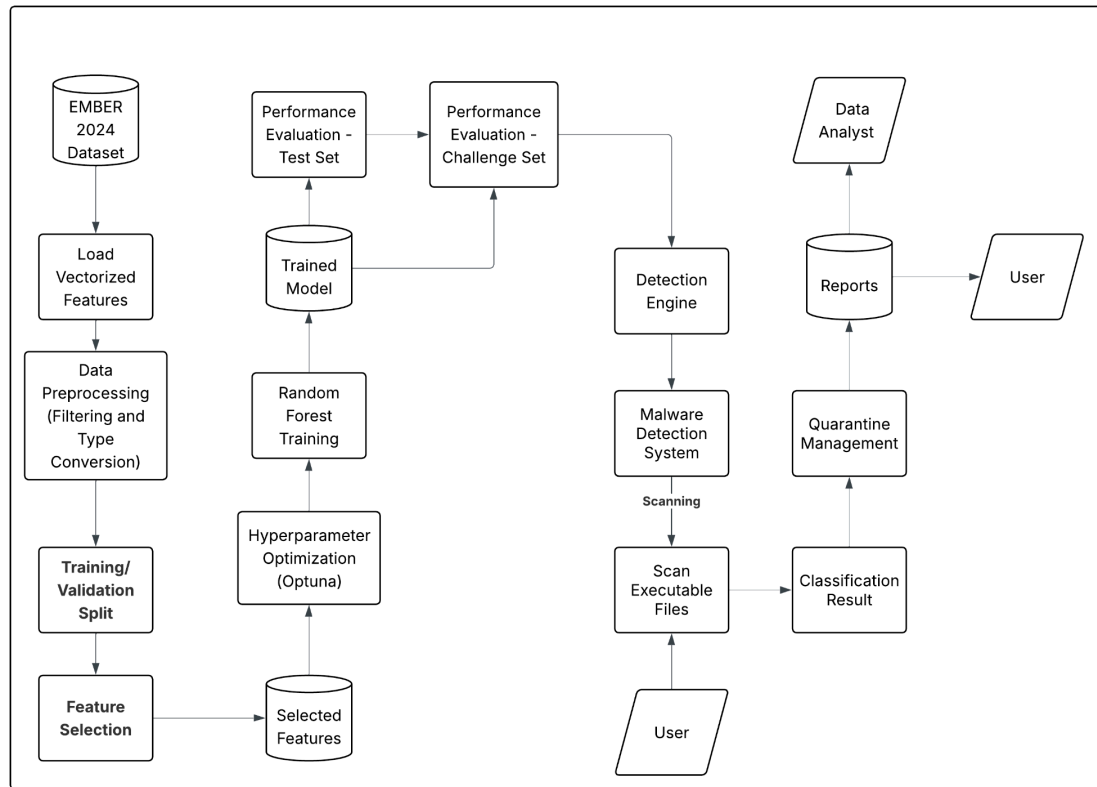


Figure 2. Data Flow Diagram

The Data Flow Diagram shown is a representation of how our system works. It illustrates the internal processes and data interactions within the malware detection system. The process begins by loading the vectorized features from the EMBER 2024 dataset. The data is then preprocessed by removing unlabeled samples and converting the data types. The same preprocessing steps are applied to the training, test, and challenge datasets. The training dataset is split into two subsets: train and validation - the training subset for training the model and the validation set for the hyperparameter optimization

(Optuna). A preliminary Random Forest is trained to compute feature importance and selecting the most informative features for model training.

After determining the optimal hyperparameters identified through Optuna, they are used to configure and train the final Random Forest model. The trained model is evaluated using the test set and challenge set to assess the model's ability to generalize and accurately detect previously unseen malware.

The trained model is then used as the detection engine for the malware detection system. Once deployed, users have the chance to interact with the malware detection system. The user can update the Random Forest Model if the system has detected there is an update available. If the user commands the system to scan unknown files, the detection engine will begin scanning for malware and classify them according to their severity status by the system's classification manager. The user can then quarantine any detected malware and delete them or ignore detection for selected items. The system generates reports summarizing each scan. Users and analysts will be able to study how accurate the system is.

Algorithm Discussion

This research uses a machine learning classification algorithm, specifically random forest classification to detect malware based on extracted features from portable executable files such as .exe .dll, .sys, .ocx, .scr and other executable

formats. The algorithm process begins with inputting data or files then applying the trained model to it to detect malware or benign files, and then outputting a classification model indicator whether the file is Malware or Benign.

The algorithm process is divided into two separated into two phases: the Training Phase and the Detection Phase. The training phase will consist of training the Random Forest Model into detecting malware consistently and efficiently. The detection phase will consist of the actual malware detection system. It will consist of the user scanning files on their device and the system will display a classification output indicating whether the file is malicious or benign.

Training Phase

Step 1: Load vectorized features from the EMBER dataset

- 350,000 PE files - Training Set loaded
- 300,000 PE files - Test Set loaded
- Challenge Set loaded

Step 2: Convert feature values into float32 format for optimization

Example:

- File: sample.exe
- → entropy = 6.23
- → size = 1.4MB
- → imports = 32

Step 3: Remove samples with undefined labels (label = -1), convert features and labels to efficient data types.

Step 4: Split training set into training and validation

Step 5: Apply Feature Selection by training a preliminary Random Forest to identify and retain the most informative features.

Step 5: Initialize hyperparameter optimization (Optuna) for optimal training of Random Forest model using the validation set.

Step 6: Configure the final Random Forest Training to the optimal parameters identified from Optuna

Step 7: Train the model using the processed training dataset where multiple decision trees are constructed using random subsets of data and features to improve classification performance and reduce overfitting

Example:

Suppose a PE file has been converted into its corresponding feature vector. The feature vector is provided to the trained Random Forest classifier. Each decision tree evaluates different feature conditions and produces an individual prediction:

Decision trees vote:

- Tree 1 → Malware
- Tree 2 → Malware
- Tree 3 → Benign
- Tree 4 → Malware
- Tree 5 → Malware

Four out of five decision trees classify the file as Malware. Through majority voting, the Random Forest produces the final classification of Malware.

Step 8: Evaluate the model using the test set and the challenge set to assess the model's generalizability

Step 9: Save the trained model for future use

Detection Phase

Step 1: Input Executable files into the System

Step 2: Extract features into an EMBER dataset format

Step 3: Select the trained feature subset and convert the features into the required input format.

Step 4: Pass features into the Random Forest model

Step 5: Predict the class label - Malware or Benign

Step 6: Display Result to User

Software Engineering Paradigm

Agile Methodology

This study implements Agile methodology as its Software Engineering paradigm. Agile methodology emphasizes iterative software development and continuous improvement through an incremental approach of functional system requirements. This approach is suitable particularly for machine learning and the integration of a machine learning model for malware detection, user interface

design and system security features which often requires frequent and repetitive testing for optimal performance and optimization.

The development process is divided into multiple sprints with each sprint focusing on a specific set of functionalities. Each iteration produces a working component of the system allowing the development group to evaluate the system, identify issues and deliver improvements through cycles.

Sprint Planning

The Sprint Planning phase establishes the sequence, scope, and objectives for each development sprint. It transforms the initial product backlog into specific, manageable tasks that can be accomplished within defined timeframes. The goal of this phase is to ensure that each sprint produces a functional output that contributes directly to the overall progress of the system.

The team begins by assessing the product backlog and determining important priorities based on project dependencies and development flow. Task estimation is accomplished by using story points to quantify work and complexity. Sprint goals are outlined clearly to fit with the project's general objectives, such as developing the Random Forest classifier, upgrading the PyQt6 interface, or

integration of the quarantine system. The workload is split among team members according to their competence to achieve effectiveness and balance.

In order to sustain cooperation over the sprint cycles, this phase also creates communication protocols and review procedures. To guarantee openness and steady growth, daily meetings, code review, and documentation upgrades are conducted on a regular basis. The team assesses the sprint's achievements, talks about the difficulties faced, and decides on enhancements for the following iteration during the review and retrospective that follows each sprint. The project maintains consistent progress while allowing for modifications depending on testing results and stakeholder input thanks to this methodical but adaptable planning methodology.

Development Sprints

Sprint 1:Dataset Handling and Feature Extraction

The first sprint involves data processing and feature engineering. The dataset containing malware and benign examples is gathered and preprocessed. Various techniques for feature extraction are used to examine PE file attributes including headers, imports, entropy, metadata, and sections needed for training the Random Forest model. Data processing is done via pandas, NumPy, pefile, and EMBER PEFeatureExtractor libraries. The trained data set is ready to be deployed using ONNX Runtime in order to provide better inference performance.

This sprint also lays the groundwork for the educational malware analysis functionality by preparing the features which will later be shown to the user.

Sprint 2: Project Setup and Environment Configuration

The second sprint is about establishing the basic environment needed for the project. It covers creating a Python virtual environment, installing dependencies including Dear PyGui, ONNX Runtime, python-magic, watchdog, psutil, pefile, and scikit-learn, as well as setting up the basic folder structure for the dataset, model, quarantine, configuration, and logging files. In addition, the team came up with the first version of the GUI interface using Dear PyGui as the basis for the graphical user interface. This step is where the documentation of the environment and architecture is done to inform future developments of the malware education tool.

Sprint 3: Model Development and Training

The third sprint centers on building and training the Random Forest classifier. The preprocessed dataset is divided into training and testing subsets to evaluate model performance objectively. Various hyperparameters are adjusted to achieve optimal accuracy, precision, and recall. Performance metrics are visualized through confusion matrices and accuracy plots using Matplotlib. Once

finalized, the trained model is serialized and stored using Joblib for future use in the application.

Sprint 4: Creation of Graphical User Interface

Fourth Sprint The GUI design and implementation will be undertaken in this sprint using Dear PyGui. GUI will act as the main interface for users where they will interact with all the functionalities of the system including Quick Scan, Full Scan, Custom Scan, quarantine, real time monitoring, activity log, and reports. The GUI is designed with the aim of offering an educational interface to the users in the sense that students will be able to see the results of malware detection, machine learning classification of files, scan reports, and basic cybersecurity knowledge from the visual feedback offered by the GUI. This sprint will focus on the creation of the interface design including navigation between interfaces.

Sprint 5: Model Integration with Graphical User Interface

During the fifth sprint, the Random Forest model that was trained and exported in ONNX Runtime format is included in the application for Dear PyGui. Each file is initially filtered by python-magic in order to check whether it is really of the given file type using magic bytes. Non-executable files are instantly ignored, and only Portable Executable (PE) files, namely .exe, .dll, and .sys, undergo further analysis. The remaining files are read by means of pefile,

necessary features are extracted, and the prediction on the presence of malware is made using the ONNX model. The results of the scan are instantly shown in color codes accompanied by explanations to teach users about malware detection and classification.

Sprint 6: Quarantine and File Management System

The sixth sprint focuses on implementing the quarantine and file management module. Detected malware files are automatically moved to a secure quarantine directory. The interface provides options to view quarantined files, restore them if necessary, or permanently delete them. Additional security measures are integrated to prevent unauthorized access or accidental file recovery. This sprint enhances the safety and integrity of the overall system by ensuring proper handling of detected threats.

Sprint 7: User Interface Enhancement and Visualization

In the seventh sprint, the focus is on developing an improved graphical user interface along with visualizations. The GUI has been improved by adding status bars, progress bar, icons, notifications, and scanning statistics for the enhancement of its usability. Visualization has been used for the purpose of displaying trends in detections, scanning results, and system statistics in an educational and easily comprehensible way. The scanning engine has been implemented using background threads so that the interface does not freeze in case of long scanning processes.

Sprint 8: Logging and Reporting Module

Sprint eight entails the development of the logging and reporting features. All scan outcomes, quarantines, monitoring, and user activities are logged for future analysis purposes. A section for reporting is developed within the Graphical User Interface (GUI), whereby users can access the scan history, activity logs, and generate report summaries. In order to provide convenience and real-time notifications regarding any events, the system incorporates the Telegram Bot notification feature. This feature allows users to receive real-time notifications regarding the malware detections and security alerts on their mobile devices. This helps users to monitor their scan activities from anywhere and deal with any potential threats immediately.

Sprint 9: Testing and Quality Assurance

The ninth sprint is dedicated to comprehensive system testing and quality assurance. Unit testing and integration testing are performed to verify that each module functions correctly. Performance testing evaluates scanning efficiency using the optimized detection pipeline, including magic-byte filtering, scan caching, background processing, and ONNX Runtime inference. Stress testing is conducted using large datasets and repeated scans to validate responsiveness and stability. The Random Forest model is evaluated using real-world malware samples to verify detection accuracy, reliability, and overall system performance.

The educational features and user interface are also assessed to ensure they effectively communicate malware detection results to users.

Sprint 10: Deployment and User Documentation

The tenth sprint finalizes the system for deployment. The project is packaged into a standalone executable using PyInstaller, allowing it to run on Windows without additional setup. A comprehensive user manual is created to guide users through installation, scanning procedures, interpreting malware detection results, and understanding the system's educational cybersecurity features. Final performance tests are conducted to ensure the application functions smoothly before official deployment and release.

Post-Launch Sprints(Sprint 11 and Beyond)

Post-launch sprints focus on continuous improvement and system maintenance. User feedback is collected and analyzed to identify potential enhancements. Future updates may include the integration of additional machine learning models, automated updates for malware signatures, and improved real-time threat monitoring. Regular retraining of the Random Forest model using new datasets ensures adaptability to evolving malware threats. These ongoing activities guarantee that the system remains effective, efficient, and up to date in protecting users against cybersecurity risks

Tool to test for accuracy

To ensure the accuracy of this study, the study employs the Python library scikit-learn for the evaluation metrics and the usage of the standard performance metrics (Accuracy, Precision, Recall, F1-Score). As well as the addition of Confusion Matrix and ROC Curve/AUC. These performance measures were added to test for the accuracy of the model and to ensure robustness and reliability of the system.

Confusion Matrix was generated to provide a detailed breakdown of predicted outcomes - true positives, true negatives, false positives, and false negatives. Additionally Receiver Operating Characteristics (ROC) Curve and the Area Under the Curve (AUC) were used to evaluate the model's performance metrics in distinguishing between malware and benign files across classification thresholds.

Population and Sampling

Dataset Population

The study uses purposive sampling through secondary data, whereby only the labeled malicious and benign PE files in the EMBER 2024 data set were chosen for supervised learning. The data set has a balanced representation of malware and benign files, hence lessening the impact of class imbalance in supervised learning.

The target population for this research study is all real world Portable Executable (PE) binaries that could either be malicious or benign in nature. PE files were selected as the target population since they form the most common executable format of Windows operating system (.exe,.dll, .sys, etc.).

The EMBER 2024 dataset (Joyce et al.) was chosen as it provides a representative sample of the broader PE file population. The full dataset contains approximately 3.2 million binary file samples. For the supervised classification task, only the labeled malicious and benign samples were retained, while the unlabeled portion was excluded as this study focuses strictly on fully supervised binary classification

This sampling approach offers several advantages: The dataset is already balanced with roughly a 50% balance split for malware and benign files which reduces class-imbalance bias; Its large scale supports robust model training and generalization testing; and its status as a standard benchmark for malware detection enables direct comparison with previous studies.

Human Evaluation Participants

As for the assessment of the system, the human subjects would be CCS students, IT experts, and professors from Pamantasan ng Lungsod ng Pasig. The reason why these people were selected is that they will be using the system, therefore, they are the best choice for providing feedback on it.

A total of 100 students will be the target respondents to determine the success of the system. Random sampling will be applied to choose these 100 student respondents to ensure that there is unbiased feedback on student knowledge and the education interface of the system. Though the main group of respondents to be used in the survey is the BS Information Technology (BSIT) under the College of Computer Studies (CCS), other students from other courses can also be chosen randomly.

Furthermore, purposive sampling will be used to identify IT experts and teachers for evaluation and analysis of the system. This will help to provide broad feedback on the usability of the system among the students and at the same time test its validity among the experts.

Data Collection Methods

The research makes use of secondary data collection for building machine learning models and primary data collection for usability and functionality analysis of the system. For secondary data collection, the EMBER 2024 dataset was used, which is an open-source benchmark repository that can be downloaded from the Elastic Malware Benchmark for Empowering Researchers (EMBER) GitHub repository. In order to make the computational process more efficient but at the same time statistically rigorous, a sample of 350,000 training samples and 300,000 testing samples was taken from vectorized features of the

dataset. The data was collected through the Ember Python library where unlabeled samples were deleted in order to conduct fully supervised binary classification. In addition, feature vectors were formatted into float32 in order to optimize the processing of the data, and the labeled data was stratified into 80% for training the model, 20% for hyperparameter optimization by means of Optuna and hold-out test.

In addition to the secondary data provided by the model, primary data will be collected in order to assess the functional, usability, and educational effectiveness of the AYAWrus system according to ISO/IEC 25010 standards. Primary data will be collected from 100 students who are the respondents through an evaluation form after their experience with the system. This will include user interface navigation, feedback provided to diagnose problems, and educational benefit from the system. In addition, qualitative and quantitative evaluation forms will be distributed to selected IT experts and educators for technical assessment of the system.

Data Analysis Procedures

The data analysis will employ machine learning analysis and techniques to evaluate the performance, efficiency, functionality and educational effectiveness of the malware detection system as well as the effectiveness of the acquired dataset - EMBER 2024. The analysis will focus on measurable metrics such as

confusion matrices Area Under the Receiver Operating Characteristic Curve (ROC-AUC), classification reports, and feature importance analysis.

The EMBER 2024 dataset was preprocessed by removing unlabeled samples to ensure only labeled benign and malicious files were included in the analysis. The remaining feature vectors were converted to appropriate and efficient numerical data types to improve computational efficiency. The dataset was then split into training and validation using stratified sampling for the usage of the validation set for the hyperparameter optimization (Optuna).

Feature selection was then performed using a preliminary Random Forest classifier trained on a subset of the training data. Feature importance scores generated by the classifier were used to identify and gather informative attributes. Features with importance values below predefined threshold were to reduce dimensionality, improve computational efficiency and retain only relevant features for the final model.

As gathered in the first step, the validation set will be used for the hyperparameter optimization. Hyper parameter optimization was conducted using the Optuna optimization framework. Multiple trials and combinations of Random Forest parameters(tree depth, number of trees, etc) were evaluated. The optimization objective prioritized maximizing recall while maintaining a minimum precision threshold of 0.85 to reduce false-positive predictions.

After the training process, probability scores determine how dangerous a file was generated by the Random Forest classifier using multiple decision thresholds. The optimal threshold was selected based on the highest F-best score ($\beta = 3$) to allow for greater emphasis on recall and reducing false negatives. This ensures that malicious files were less likely to be incorrectly classified as benign.

The optimized model was evaluated on the previously unseen testing dataset and challenge dataset. Performance, functionality and ability to generalize new threats were measured using the standard classification metrics using accuracy, precision, recall, F1-score and Area Under the Receiver Operating Characteristic Curve (ROC-AUC). A confusion matrix and ROC curve were generated to analyze the model's ability to differentiate between benign and malicious files and the ability to identify between false positives and false negatives.

Every step mentioned above was all implemented in Python. The sci-kit learn library was used for machine learning algorithms and evaluation metrics, Optuna for hyperparameter optimization, Joblib for model serialization and matplotlib, and seaborn for visualization. These python library plugins allow for a straightforward and clear presentation of the machine learning analysis.

Ethical Considerations

This study follows the standard ethical and data protection principles in the development and evaluation of the malware detection system. All datasets gathered and utilized in this research were obtained from publicly available and legally accessible sources. No personal or private information was collected nor analyzed in this research.

To ensure the safety of handling malicious files, precautionary measures were implemented to ensure the safety of the researcher in the computing environment. Malicious and benign files were managed in an isolated and controlled environment to prevent system execution or compromise. These files were ensured to be in a non-activation state to prevent unintended execution.

In addition, the study complies with data privacy principles by ensuring no scanned files are stored or transmitted externally or used for model training with proper authorization or consent. This ensures the users' confidentiality and privacy.

The study also acknowledges that, similar to all malware detection systems, the proposed system may not be able to achieve a hundred percent detection in accuracy and performance as malware is ever-evolving and some malware may evade detection.

Any information of any participants involved in the study will not be held nor used by the researchers during this study. Any information collected will be for the benefit of the study - feedback and recommendations about the feasibility, functionality, and educational effectiveness of this system.

Reference List

- A. Awasthi, M. Ahsan, V. D and D. R. S, "SecureML: Machine Learning-Powered Malware Detection," 2024 Second International Conference on Advances in Information Technology (ICAIT), Chikkamagaluru, Karnataka, India, 2024, pp. 1-9, doi: 10.1109/ICAIT61638.2024.10690740. <https://ieeexplore.ieee.org/document/10690740>
- Allix, K., Bissyandé, T. F., Jérôme, Q., Klein, J., State, R., & Le Traon, Y. (2014). Large-scale machine learning-based malware detection: Confronting the “10-fold cross validation” scheme with reality. In Proceedings of the 4th ACM Conference on Data and Application Security and Privacy (pp. 163–174). ACM. <https://doi.org/10.1145/2557547.2557567>
- Al-Janabi, M., & Altamimi, A. M. (2022, May). A comparative analysis of machine learning techniques for classification and detection of malware. ResearchGate. https://www.researchgate.net/publication/360485697_A_Comparative_Analysis_of_Machine_Learning_Techniques_for_Classification_and_Detection_of_Malware.ResearchGate
- Anderson, H. S., & Roth, P. (2018). *EMBER: An open dataset for training static PE malware machine learning models*. arXiv. <https://arxiv.org/abs/1804.04637>
- Bejjam, J., et al. (2023). *Unveiling the veiled: Unmasking fileless malware through memory forensics and machine learning*. *International Journal on*

Recent and Innovation Trends in Computing and Communication, 11(9), 3691–3700. <https://doi.org/10.17762/ijritcc.v11i9.9592>.

<https://ijritcc.org/index.php/ijritcc/article/view/9592>

Borate, V., Adsul, A., Gaikwad, A., Mhetre, A., & Dicholkar, S. (2024). Analysis of malware detection using various machine learning approach. *International Journal of Advanced Research in Science, Communication and Technology*, 4(2), doi:10.48175/IJAR SCT-22159 <https://ijarsct.co.in/Paper22159.pdf>

Ceschin, F., Botacina, M., Gomes, H. M., Pinagé, F., Oliveira, L. S., & Grégio, A. (2022). Fast & Furious: Modelling malware detection as evolving data streams. *Expert Systems with Applications* [Preprint]. <https://arxiv.org/abs/2205.1231>

Chaudhary, D., Bakshi, K., & Jha, P. K. (2024). Detection of malware and adware using machine learning: A practical approach [Preprint]. Research Square, doi:10.21203/rs.3.rs-4219382/v1. <https://www.researchsquare.com/article/rs-4219382/v1>

Fauzan, M., Mawardi, C., & Asgawanti, E. (2025, July 9). *Implementation of a web-based malware analysis system with random forest integration*. *International Journal of Computer and Information System (IJCIS)*, 6(3), 200–208. <https://doi.org/10.29040/ijcis.v6i3.242>. https://www.researchgate.net/publication/394265162_Implementation_of_a_Web-Based_Malware_Analysis_System_With_Random_Forest_Integration/citations

- Funcion, D. G. D., & Quisumbing, L. A. (n.d.). *Am I secured: A computer virus awareness among BSIT students*. Leyte Normal University.
https://www.researchgate.net/publication/336406476_Am_I_Secured_A_Computer_Virus_Awareness_among_BSIT_Students
- G. Koç and M. Aydos, "Trustworthy scrum: Development of secure software with scrum," 2017 International Conference on Computer Science and Engineering (UBMK), Antalya, Turkey, 2017, pp. 244-249, doi: 10.1109/UBMK.2017.8093383.
<https://ieeexplore.ieee.org/document/8093383>
- Garcia, F. C. C., & Muga, F. P., II. (2016). Random forest for malware classification. arXiv preprint arXiv:1609.07770.
<https://doi.org/10.48550/arXiv.1609.07770>
- Gundoor, T. K., Sridevi, & Mulimani, R. (2024). AI-based solutions for malware detection and prevention. In *Machine intelligence applications in cyber-risk management* (pp. 107–134). IGI Global.
<https://doi.org/10.4018/979-8-3693-7540-2.ch006>
- H. K. Dam, T. Tran, J. Grundy, A. Ghose and Y. Kamei, "Towards Effective AI-Powered Agile Project Management," 2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), Montreal, QC, Canada, 2019, pp. 41-44, doi: 10.1109/ICSE-NIER.2019.00019.
<https://ieeexplore.ieee.org/document/8805739>

- Indykov, V., Strüber, D., & Wohlrab, R. (2025). Architectural tactics to achieve quality attributes of machine-learning-enabled systems: A systematic literature review. *Journal of Systems and Software*, 223, 112373. <https://doi.org/10.1016/j.jss.2025.112373>.
<https://www.sciencedirect.com/science/article/pii/S016412122500041X>
- Hasbini, M. A. (2024, October 2). Finding a needle in a haystack: Machine learning at the forefront of threat hunting research. Securelist. <https://securelist.com/machine-learning-in-threat-hunting/114016>
- K. Vaidhyathan, A. Chandran, H. Muccini and R. Roy, "Agile4MLS—Leveraging Agile Practices for Developing Machine Learning-Enabled Systems: An Industrial Experience," in *IEEE Software*, vol. 39, no. 6, pp. 43-50, Nov.-Dec. 2022, doi: 10.1109/MS.2022.3195432. <https://ieeexplore.ieee.org/document/9845437>
- Kamdan, Y. P., Munzi, R. S., Mustafa, A. B., & Kharisma, I. L. (2025). Static malware detection and classification using machine learning: A random forest approach. *Engineering Proceedings*, 107(1), 76. <https://doi.org/10.3390/engproc2025107076>
- Khaled, M. A., Saleh, M., Nasser, Y., Salah, F., Hani, A., Ahmed, M. E., Abdullah, A., Adell, A., Abdullah, M., & Alsakkaf, N. A. (2025). A study on the cyber attack awareness among students: University of Science and Technology case study. *Journal of Science and Technology*, 30(8), 65–73. <https://doi.org/10.20428/jst.v30i8.2918>
<https://journals.ust.edu/index.php/JST/article/view/2918>

- Kikelomo, A. A., Asafe, Y. N., & Oludayo, O. I. (2023). *Malware detection system using mathematics of random forest classifier. International Journal of Advances in Scientific Research and Engineering (IJASRE)*, 9(3), 45–53.
<https://doi.org/10.31695/IJASRE.2023.9.3.6>.
<https://ijasre.net/index.php/ijasre/article/view/1664>
- Madamidola, O. A., Ngobigha, F., & Ez-zizia, A. (2024). *Detecting new obfuscated malware variants: A lightweight and interpretable machine learning approach*. <https://arxiv.org/pdf/2407.07918>
- Manzil, H. H. R., & Manohar Naik, S. (2023). Android malware category detection using a novel feature vector-based machine learning model. *Cybersecurity*, 6, 6. <https://doi.org/10.1186/s42400-023-00139-y>.
- Manzoor, M., & Arora, B. (2023, May 17). *Framework for detection of malware using random forest classifier*. In *Lecture Notes in Electrical Engineering* (pp. 727–740). Springer. https://doi.org/10.1007/978-981-99-0601-7_56.
https://www.researchgate.net/publication/370831534_Framework_for_Detection_of_Malware_Using_Random_Forest_Classifier
- Matsobane, N. O., & Mokwena, S. (2025). *Malware detection using a random forest method trained on a balanced synthetic dataset*. *Science, Engineering and Technology*, 5(1).
https://www.researchgate.net/publication/390276210_Malware_Detection_Using_a_Random_Forest_Method_Trained_on_a_Balanced_Synthetic_Dataset

Morales, J. A. (2014, April 21). *A new approach to prioritizing malware analysis*.

Software Engineering Institute Blog. Carnegie Mellon University.

<https://www.sei.cmu.edu/blog/a-new-approach-to-prioritizing-malware-analysis>

Omowunmi, A. O. T., Adedayo, L. Y., & Kareem, B. (2023). Cybersecurity risk

management in agile development: Protecting data and system.

International Journal of Science and Research Archive, doi:

10.30574/ijrsra.2023.8.1.0188.

<https://ijrsra.net/content/cybersecurity-risk-management-agile-development-protecting-data-and-system>

R. Ranawana and A. S. Karunananda, "An Agile Software Development Life

Cycle Model for Machine Learning Application Development," 2021 5th

SLAAI International Conference on Artificial Intelligence (SLAAI-ICAI),

Colombo, Sri Lanka, 2021, pp. 1-6, doi:

10.1109/SLAAI-ICAI54477.2021.9664736.

<https://ieeexplore.ieee.org/document/9664736>

Reddy, R. K. S., Kumar, K. S., Rao, E. S. N., & RajaRajeswari, P. (2023).

Enhancing cybersecurity through random forests: A comprehensive

analysis of malware detection and intrusion detection. In *Proceedings of*

the 2023 Third International Conference on Ubiquitous Computing and

Intelligent Information Systems (ICUIS) (pp. 478–482). IEEE.

<https://doi.org/10.1109/ICUIS60567.2023.00085>.

<https://ieeexplore.ieee.org/document/10506014>

- Shahbazi, Z., Jalali, R., & Molaevand, M. (2025). AI-based phishing detection and student cybersecurity awareness in the digital age. *Big Data and Cognitive Computing*, 9(8), 210. <https://doi.org/10.3390/bdcc9080210>
<https://www.mdpi.com/2504-2289/9/8/210>
- Siavvas, M., Kehagias, D., Tzovaras, D., & Gelenbe, E. (2021). A hierarchical model for quantifying software security based on static analysis alerts and software metrics. *Software Quality Journal*, 29(2), 431–507. <https://doi.org/10.1007/s11219-021-09555-0>.
<https://dl.acm.org/doi/10.1007/s11219-021-09555-0>
- Silva, J., & Hernandez-Alvarez, M. (2023). *Deployment of ransomware detection using dynamic analysis and machine learning*. In A. Moallem (Ed.), *Human factors in cybersecurity* (Vol. 91, pp. [page range if known]). AHFE International. <https://doi.org/10.54941/ahfe1003714>.
https://openaccess.cms-conferences.org/publications/book/978-1-958651-67-4/article/978-1-958651-67-4_0
- Syed Suhaila, S., & Sundara Krishnan, K. (2024, April). Improving Windows malware detection using the Random Forest algorithm and multi-view analysis. *International Journal of Software Engineering and Knowledge Engineering*, 34(2), 1–31. <https://doi.org/10.1142/S0218194024500086>.
- Zamri, W. F. H. B. W., & Dhakal, M. (2025). *Malware detection using the concept of random forest algorithm*. *Journal of Science & Technology* (Paper No. 12). SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5331142

- Zamri, W. F. H. B. W., & Dhakal, M. (2025). *Malware detection using the concept of random forest algorithm. Journal of Science & Technology* (Paper No. 12). SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5331142
- Goliath, S., Tsibolane, P., & Snyman, D. (2026). Exploring the cybersecurity-resilience gap: An analysis of student attitudes and behaviors in higher education. In H. Venter, M. Eloff, J. Eloff, R. Botha, M. Loock, & U. Mushtaq (Eds.), *Information and Cyber Security (ISSA 2024, Communications in Computer and Information Science, Vol. 2661)*. Springer. https://doi.org/10.1007/978-3-032-09660-9_19
- Maity, S., & Deroy, A. (2024). Human-centric eXplainable AI in education. arXiv preprint arXiv:2410.19822. <https://arxiv.org/pdf/2410.19822>
- Paterakis, G., Castellani, A., Papoutsoglou, G., Rodemann, T., & Tsamardinos, I. (2025). A comprehensive perspective on explainable AI across the machine learning workflow. arXiv preprint arXiv:2508.11529. <https://arxiv.org/pdf/2508.11529>
- Berrios, S., Leiva, D., Olivares, B., Allende-Cid, H., & Hermosilla, P. (2025). Systematic review: Malware detection and classification in cybersecurity. *Applied Sciences*, 15(14), 7747. <https://doi.org/10.3390/app15147747>
- Seneviratne, H. A., Thenabadu, M., & Wijerathne, W. M. G. K. (2022). An exploratory study on information security vulnerabilities in higher education: Case of University of Vocational Technology, Sri Lanka. *International Journal of Research and Innovation in Social Science*, 6(6), 399–403.

Vasović, D., Janačković, G., Vranjanac, Ž., Stamenković, S., & Vasović, B. (2026). Enhancing CIA triad—Confidentiality, integrity and availability in educational information systems through next-generation ISO/IEC 27001:2022-aligned security model. *Applied Sciences*, 16(12), 6260. <https://doi.org/10.3390/app16126260>